SENG 34213 / FINAL DELIVERABLES / FOOD ORDERING SYSTEM (FOSS)

Performance Test Results — Food Ordering System (FOSS)

FOSS-specific performance testing specification and evaluation framework. Scenario plan is fully reproducible; strictly notes that NO actual load, stress, latency, or throughput measurements were executed against shared infrastructure.

DOCUMENT STATUS: NOT TESTED — MEASUREMENTS REQUIRED

1 Testing result and evidence boundary

Required Field	Recorded Subsystem Result
Performance testing tool	Not executed / not verified. k6 is the proposed tool; zero test runs executed in CI.
Target environment	Not provisioned / not verified. Requires isolated staging cluster and disposable DB.
Tested revision	develop | 9ac1e6f (baseline commit for test architecture)
Concurrent users	0 observed. Proposed scenario targets 100 concurrent guest sessions (SRS NFR-01).
Test duration	0 seconds observed. Proposed scenario: 5-minute ramp load test.
Throughput	Not measured. Zero load benchmarks executed.
Response-time p50 / p95	Not measured. Target: p95 <= 500 ms under 100 concurrent users.
HTTP error rate	Not measured. Target: < 0.1% non-validation HTTP errors.
RabbitMQ event propagation	Not measured in live load. Unit/integration mocks verify consumer parsing only.
SSE client update latency	Not measured under load. Target: inter-service event arrival <= 1000 ms.
Polling fallback behavior	Functionally verified in frontend unit tests (switch to 5s polling on SSE failure).
Database performance	Not measured. No query profiling under concurrent load.

2 Relevant Objectives (Targets are NOT Results)

• **NFR-01 (API Latency):** 95% of guest API requests completed within **500 ms** at 100 concurrent active room sessions.
• **NFR-03 (Real-Time Freshness):** Inter-service state changes (kitchen order preparation, worker assignment) propagated to guest SSE client within **1.0 second**.
• **NFR-05 (Order Durability):** Idempotent order placement survives network drops without double-charging or message loss.
• **NFR-14 (Real-Time Scalability):** FOSS backend maintains **300 concurrent WebSockets/SSE connections** without memory exhaustion.

3 Planned FOSS Performance Test Scenarios (Reproducible Plan)

Scenario ID	Target Flow	Endpoints / Protocol	Load Profile	Success Criteria
SC-PERF-01	**Guest QR Activation**	POST /guest-sessions/qr	50 req/sec burst	p95 < 300 ms, 0% 5xx errors
SC-PERF-02	**Menu Catalog Browsing**	GET /menu, GET /menu/windows	100 concurrent users	p95 < 200 ms (cached)
SC-PERF-03	**Food Order Placement**	POST /food-orders (idempotent)	20 orders/sec ramp	p95 < 500 ms, 0 duplicates
SC-PERF-04	**Order Status & History**	GET /food-orders, GET /:id	80 concurrent users	p95 < 250 ms, room isolated
SC-PERF-05	**Service Request Creation**	POST /service-requests	15 req/sec ramp	p95 < 400 ms, auto-bound room
SC-PERF-06	**Complaint Submission**	POST /complaints	10 req/sec ramp	p95 < 400 ms, CMP ref assigned
SC-PERF-07	**Realtime SSE Streaming**	GET /realtime/events/stream	300 concurrent hold	Stable heap memory < 256MB
SC-PERF-08	**Optional Redis Cache Test**	GET /menu (Redis ON vs OFF)	100 concurrent users	Evaluate throughput gain

4 Controlled Execution Prerequisites & Safeguards

• **Never Test Against Production or Development Neon DB:** Load testing against shared branches risks rate limiting, connection exhaustion, and data corruption. HealthService guards will intentionally abort tests targeting unauthorized branches.
• **Dedicated Disposable Schema:** Performance runs must execute against a dedicated, isolated test database populated with synthetic guest sessions.
• **Mock Upstream Services:** Upstream subsystems (FDS, KMS, WKMS, MAD) must use mock adapters during load testing to avoid artificial upstream bottlenecks or cross-team disruption.
• **Resource Metrics:** Record CPU, memory, event loop latency, and connection pool saturation during any future staging execution.